

III MODULE

There are many ways to design an algorithm for a given problem.

The following is a list of several popular design approaches.

- ⇒ Divide and Conquer
- ⇒ Greedy Approach
- ⇒ Dynamic Programming
- ⇒ Branch and Bound, and
- ⇒ Backtracking.

Divide and Conquer.

⇒ Divide the original problem into a set of sub problems.

⇒ Solve every sub problem individually recursively.

⇒ Combine the solutions of the sub problems into a solution of the whole original problem.

Greedy Approach

⇒ Greedy Algorithm seek to optimize a function by making choices locally.

⇒ The result is a good solution but not necessarily the best one.

⇒ It doesn't always guarantee the optimal solution, however generally produces solutions that are very close to the optimal solution.

Dynamic Programming

⇒ Generally used for optimization problems.

⇒ Techniques for efficiently computing recurrences by storing partial results.

⇒ It is a method of solving problems exhibiting the properties

of overlapping subproblems and optimal substructure.

Branch and Bound

⇒ It is a method for global optimization.

⇒ In a branch and bound algorithm, a given subproblem, which cannot be bounded has to be divided into at least two new restricted subproblems.

Backtracking Algorithms.

⇒ Backtracking algorithms try each possibility until they find the right one.

⇒ During the search, if an alternative does not work, the search backtracks to the choice point, and tries the next alternative.

⇒ If there are no more choice points, the search fails.

Divide and Conquer

Divide and Conquer is a technique for designing algorithms.

It consists of decomposing the instance to be solved into a number of smaller subinstances of the same problem, solving successively and independently each of these subinstances and combining the subsolutions thus obtained to obtain the solution of the original instance.

Divide and Conquer paradigm consists of following major steps

Divide

Divide the problem into several subproblems that are similar to the original problem but smaller

in size.

Conquer

Solve the subproblems recursively.

Combine

Combine the solutions of subproblems to create a solution to the original problem.

So divide and conquer strategy solves a problem by

⇒ Breaking it into subproblems that are themselves smaller instances of the same type of problem.

⇒ Recursively solving these subproblems.

⇒ Appropriately combining these answers.

Strassen's Matrix Multiplication

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} * \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$$

Let $\begin{bmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{bmatrix}$ is the

resultant matrix produced after multiplication between two matrices

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \cdot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix}$$

The resultant matrix is produced in classical matrix multiplication as follows:

$$c_{11} = a_{11}b_{11} + a_{12}b_{21}$$

$$c_{12} = a_{11}b_{12} + a_{12}b_{22}$$

$$c_{21} = a_{21}b_{11} + a_{22}b_{21}$$

$$c_{22} = a_{21}b_{12} + a_{22}b_{22}$$

Using Strassen's matrix multiplication, the resultant matrix is obtained as follows:-

$$P_1 = (a_{11} + a_{22})(b_{11} + b_{22})$$

$$P_2 = (a_{21} + a_{22})b_{11}$$

$$P_3 = (b_{12} - b_{22})a_{11}$$

$$P_4 = (b_{21} - b_{11})a_{22}$$

$$P_5 = (a_{11} + a_{12})b_{22}$$

$$P_6 = (a_{21} - a_{11})(b_{11} + b_{12})$$

$$P_7 = (a_{12} - a_{22})(b_{21} + b_{22})$$

$$c_{11} = P_1 + P_4 - P_5 + P_7$$

$$c_{12} = P_3 + P_5$$

$$c_{21} = P_2 + P_4$$

$$c_{22} = P_1 + P_3 - P_2 + P_6$$

Comparison b/w classical and Strassen's matrix multiplication

→ In classical matrix multiplication it takes 8 multiplication to produce

the resultant matrix

So the recurrence equation

is $T(n) = 8T(n/2) + n^2$

$$a = 8 \quad b = 2 \quad f(n) = n^2$$

$$n \log_b a = n \log_2 8 = n^3$$

$$\underline{f(n) < n \log_b a}$$

$$f(n) = O(n \log_b a - \epsilon)$$
$$= O(n^2)$$

$$T(n) = O(n \log_b a)$$
$$= \underline{\underline{O(n^3)}}$$

But in Strassen's matrix multiplication, it takes only 7 multiplications to produce the result.

So the recurrence equation is

$$T(n) = 7T(n/2) + n^2$$

$$a = 7 \quad b = 2 \quad f(n) = n^2$$

$$n \log^3 n = n \log^3 7 = n^{2.80}$$

$$\underline{f(n) < n \log^3 n}$$

$$\begin{aligned} f(n) &= O(n \log^3 n - \epsilon) \\ &= O(n^2) \end{aligned}$$

$$\begin{aligned} T(n) &= O(n \log^3 n) \\ &= O(n^{2.81}) \end{aligned}$$

So we can conclude that the Strassen's matrix multiplication outperforms the classical matrix multiplication.

Example

Verify Strassen's matrix multiplication algorithm for the matrix

$$A = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad B = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$$

Ans

$$\begin{aligned} P_1 &= (a_{11} + a_{22})(b_{11} + b_{22}) \\ &= (1 + 1)(2 + 5) \\ &= 14 \end{aligned}$$

$$P_2 = (a_{21} + a_{22}) b_{11}$$

$$= (1+1) 2$$

$$= 4$$

$$P_3 = (b_{12} - b_{22}) a_{11}$$

$$= (3-5) 1$$

$$= -2$$

$$P_4 = (b_{21} - b_{22}) a_{22}$$

$$= (4-2) 1$$

$$= 2$$

$$P_5 = (a_{11} + a_{12}) b_{22}$$

$$= (1+1) 5$$

$$= 10$$

$$P_6 = (a_{21} - a_{11}) (b_{11} + b_{12})$$

$$= (1-1) (2+3)$$

$$= 0$$

$$P_7 = (a_{12} - a_{22}) (b_{21} + b_{22})$$

$$= (1-1) (4+5)$$

$$= 0$$

$$C_{11} = P_1 + P_4 - P_5 + P_7$$

$$= 14 + 2 - 10 + 0$$

$$= 6$$

$$C_{12} = P_3 + P_5$$

$$= -2 + 10$$

$$= 8$$

$$C_{21} = P_2 + P_4$$

$$= 4 + 2$$

$$= 6$$

$$C_{22} = P_1 + P_3 - P_2 + P_6$$

$$= 14 - 2 - 4 + 0$$

$$= 8$$

$$\begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$

So

$$\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 6 & 8 \\ 6 & 8 \end{bmatrix}$$

MERGE SORT

This algorithm was invented by John Von Neumann in 1945.

It follows the divide and conquer paradigm.

It works as follows:-

Divide

Divide n elements to be sorted into two subsequences of $n/2$ elements.

Conquer

Sort the two subsequences recursively.

Combine

Merge the two sorted subsequences to produce the sorted answer.

Algorithm

MergeSort(A, p, r)

if $p < r$

$q \leftarrow \lfloor (p+r)/2 \rfloor$

MergeSort(A, p, q)

MergeSort($A, q+1, r$)

Merge(A, p, q, r).

Merge(A, p, q, r)

$n_1 \leftarrow q - p + 1$

$n_2 \leftarrow r - q$

create arrays $L[1..n_1+1]$ and $R[1..n_2+1]$

for $i \leftarrow 1$ to n_1

$L[i] \leftarrow A[p+i-1]$

for $j \leftarrow 1$ to n_2

$R[j] \leftarrow A[q+j]$

$L[n_1+1] \leftarrow \infty$

$R[n_2+1] \leftarrow \infty$

$i \leftarrow 1$

$j \leftarrow 1$

for $k \leftarrow p$ to r

if $L[i] \leq R[j]$

$A[k] \leftarrow L[i]$

$i \leftarrow i + 1$

else

$A[k] \leftarrow R[j]$

$j \leftarrow j + 1$

Performance Analysis

Divide

The divide step just computes the middle of the subarray, which takes constant time $O(1)$.

Conquer

We recursively solve the subproblems, each of size $n/2$ which contributes $2T(n/2)$ to the running time.

Combine

Merge procedure on an n -element subarray takes $O(n)$ time.

$$\therefore f(n) = O(n \log n)$$

For best case, average case and worst case merge sort have the recurrence equation

$$T(n) = \begin{cases} O(1) & \text{if } n=1 \\ 2T(n/2) + n & \text{if } n>1 \end{cases}$$

$$T(n) = 2T(n/2) + n.$$

$$a=2 \quad b=2 \quad f(n)=n.$$

$$n^{\log_b a} = n^{\log_2 2} = n$$

$$f(n) = n^{\log_b a}$$

$$\begin{aligned} f(n) &= O(n^{\log_b a}) \\ &= O(n) \end{aligned}$$

$$\begin{aligned} T(n) &= O(n^{\log_b a} \log n) \\ &= \underline{\underline{O(n \log n)}} \end{aligned}$$

Merge Sort - Simple Example.

Merge Sort (15, 10, 5, 20) =

$$p=1, r=4 \therefore MS(A, p, q) \Rightarrow MS(A, 1, 4)$$

$$\boxed{\begin{array}{l} MS(A, 1, 4) \\ \{15, 10, 5, 20\} \end{array}}$$

$$\Rightarrow \begin{array}{l} p < r \\ q \Rightarrow \left\lfloor \frac{p+r}{2} \right\rfloor \\ = \left\lfloor \frac{1+4}{2} \right\rfloor \\ = 2 \end{array}$$

$$\boxed{\begin{array}{l} MS(A, p, q) \\ MS(A, 1, 2) \\ \{15, 10\} \end{array}}$$

$$\begin{array}{l} < 2 \\ \Leftarrow \\ q = \left\lfloor \frac{1+2}{2} \right\rfloor \\ = 1 \end{array}$$

$$\boxed{\begin{array}{l} MS(A, p, q) \\ MS(A, 1, 1) \\ \{15\} \end{array}}$$

$p \neq r$

$$\boxed{\begin{array}{l} MS(A, q+r) \\ MS(A, 2, 2) \\ \{10\} \end{array}}$$

$p \neq r$

$$\boxed{\begin{array}{l} Merge(A, p, q, r) \\ \Rightarrow Merge(A, 1, 1, 2) \\ \{10, 15\} \end{array}}$$

$$\boxed{\begin{array}{l} MS(A, q+r) \\ MS(A, 3, 4) \\ \{5, 20\} \end{array}}$$

$$\begin{array}{l} 3 < 4 \\ \Leftarrow \\ q = \left\lfloor \frac{3+4}{2} \right\rfloor \\ = 3 \end{array}$$

$$\boxed{\begin{array}{l} MS(A, p, q) \\ MS(A, 3, 3) \\ \{5\} \end{array}}$$

$p \neq r$

$$\boxed{\begin{array}{l} MS(A, q+r) \\ MS(A, 4, 4) \\ \{20\} \end{array}}$$

$p \neq r$

$$\boxed{\begin{array}{l} Merge(A, p, q, r) \\ Merge(A, 3, 3, 4) \\ \{5, 20\} \end{array}}$$

$$\boxed{\begin{array}{l} Merge(A, 1, 2, 4) \\ \{5, 10, 15, 20\} \end{array}}$$

$$\Rightarrow \underline{\underline{\{5, 10, 15, 20\}}}$$

GREEDY STRATEGY

Greedy Algorithms are simple and straight forward.

Greedy algorithms solve problems by making choice that seems best at the particular moment.

Many optimization problems that have no efficient solution can be solved using greedy algorithms.

Greedy algorithms may provide a solution that is close to optimal for these kinds of problems.

A greedy algorithm works if a problem exhibits the following two properties.

- 1- Greedy choice Property
- 2- Optimal Substructure.

Greedy choice Property

A Globally optimal solution can be arrived by making a locally optimal solution.

Optimal Substructure

Optimal solutions contain optimal sub solutions.

The greedy method suggests that one can devise an algorithm that works in stages, considering one input at a time.

At each stage, a decision is made regarding whether a particular input is in an optimal solution.

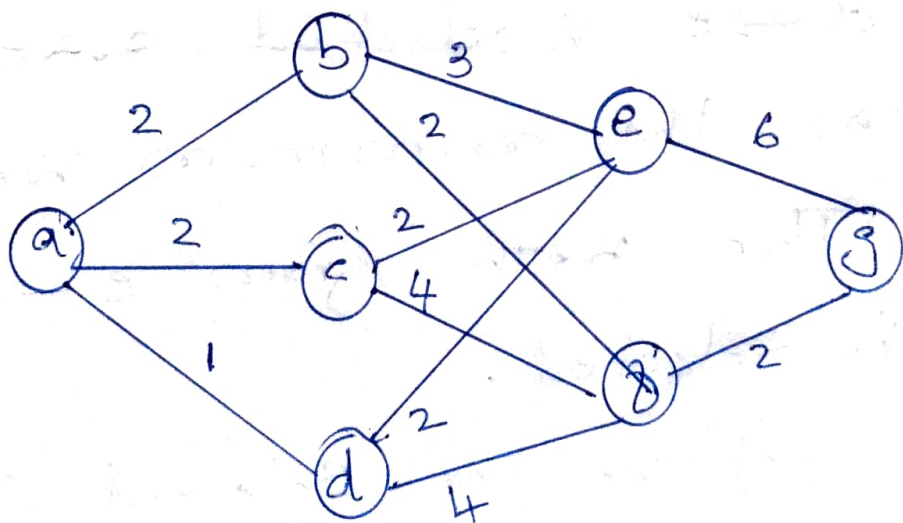
This is done by considering the inputs in an order determined by some selection procedure.

If the inclusion of the next input into the partially constructed optimal solution will result in an infeasible solution, then this input is not added to the partial solution.

Otherwise it is added.

Example

We want to find out the shortest path from a to g in the following graph using greedy strategy.



Let $SC(x)$ be the minimum distance required to reach 'g' from a node 'x'.

Greedy Algorithm will work in stages.

At the first stage it checks path from a.

[i.e., $a \rightarrow b$, $a \rightarrow c$, $a \rightarrow d$]

Path with minimum weight will be selected first

i.e., $a \rightarrow d$ is selected.

Then in the next stage path from d is checked.

[i.e. $d \rightarrow e$, $d \rightarrow g$]

$d \rightarrow e$ is selected, since it has the minimum weight.

Then path from e $\rightarrow$ g is selected.

Here the selected path is $a \rightarrow d \rightarrow e \rightarrow g$. and the weight is

$$1 + 2 + 6 \Rightarrow 9$$

But it is not an optimal solution.

Here we get only near optimal solution.

Control Abstraction for Greedy Method

Greedy(a, n)

$soln = \phi$

for $i = 0$ to n

$s = \text{select}(a);$

if feasible($soln, s$)

$soln = \text{Union}(soln, s)$

else

reject(s)

Return $soln$.

Here a selection of solution from the given input domain is performed using $s = \text{select}(a)$.

Then feasibility of the solution is checked using feasible (solution, s).

If it is feasible, it is added to the solution using union operation $\text{union}(\text{solution}, s)$.

Otherwise it is rejected.

Fractional Knapsack Problem.

Given n objects with weight w_i and value v_i and a knapsack which can hold at most weight W .

The objective is to obtain the filling of knapsack, that maximize the total profit.

Since the knapsack has capacity W , total weight of all the chosen items must be at most W .

Mathematically, we can represent

$$\begin{aligned} \text{it } \infty \quad & \text{Max } \sum_{1 \leq i \leq n} x_i V_i \\ \text{Sub. to } & \sum_{1 \leq i \leq n} w_i x_i \leq W, \text{ where} \\ & 0 \leq x_i \leq 1 \text{ and } 1 \leq i \leq n. \end{aligned}$$

Knapsack problem can be of two types.

⇒ 0/1 knapsack problem, and

⇒ Fractional knapsack problem.

0/1 Knapsack Problem

Here each item to be packed must take as whole or not.

i.e., items may not be broken into smaller pieces

Fractional Knapsack Problem

Here fractions of items can be taken rather than having to make a binary [0-1] choice for each item.

Algorithm

Fractional-knapsack (n, v, W)

for $i \leftarrow 1$ to n

$x[i] \leftarrow 0$

Weight $\leftarrow 0$

While Weight $< W$.

$i \leftarrow$ best remaining object

If Weight + $w[i] \leq W$

$x[i] \leftarrow 1$

Weight $\leftarrow$ Weight + $w[i]$

else

$x[i] \leftarrow \frac{W - \text{Weight}}{w[i]}$

Weight $\leftarrow$ Weight + $[x[i] * w[i]]$

Here the best remaining object can be selected three ways. They are

⇒ Lightest Weight

Object with least weight is selected first.

⇒ Most Valuable

Object with high value is selected first.

Value / Weight ratio

Object with high value / weight value is selected first.

Example

Consider the following instance of knapsack problem, $n=5$, $W=100$

$$(W_1, W_2, \dots, W_5) = (10, 20, 30, 40, 50)$$

$$(V_1, V_2, \dots, V_5) = (20, 30, 66, 40, 60)$$

Solution

Lightest Weight

for $i \leftarrow 1$ to n
 $i \leftarrow 1$ to 5

$$x[i] \leftarrow 0$$

$$x[i] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$\text{weight} \leftarrow 0$$

$$\text{while } \text{weight} < W$$

$$\text{weight} < 100$$

$i \leftarrow$ best remaining object

Here object with least weight is selected as best remaining object.

$i \leftarrow 1$

if $\text{Weight} + w[i] \leq W$

$$0 + 10 < 100 \Rightarrow \text{True}$$

$x[i] \leftarrow 1$

$$\text{Weight} \leftarrow 0 + 10 = 10$$

$10 < 100 \Rightarrow \text{True}$

$i \leftarrow 2$

if $10 + 20 \leq 100 \Rightarrow \text{Yes}$

$x[2] \leftarrow 1$

$$\text{Weight} \leftarrow 10 + 20 = 30$$

$30 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

if $30 + 30 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

$$\text{Weight} \leftarrow 30 + 30 = 60$$

$$x[i] \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$x[i] \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$x[i] \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

$$\underline{60 < 100} \Rightarrow \text{True.}$$

$$i \leftarrow 4$$

$$y \quad 60 + 40 \leq 100 \Rightarrow \text{Yes}$$

$$a[4] \leftarrow 1$$

$$\text{Weight} = 60 + 40 \Rightarrow 100$$

$$\underline{100 < 120} \Rightarrow \text{No}$$

exit

$a[1]$	1
$[2]$	1
3	1
4	1
25	0

Selected objects are 1, 2, 3 and
4th object

$$\text{Total Value} = 20 + 30 + 66 + 40$$

$$\Rightarrow 156$$

Most Valuable

Here object with highest value
is selected as the best remaining
object.

for $i \leftarrow 1$ to 5

$x[i] \leftarrow 0$

$x[1] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Weight $\leftarrow 0$

$0 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

if $30 + 0 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

Weight $\leftarrow 30 + 0 \Rightarrow 30$

$30 < 100 \Rightarrow \text{True}$

$i \leftarrow 5$

if $30 + 50 \leq 100 \Rightarrow \text{Yes}$

$x[5] \leftarrow 1$

Weight $\leftarrow 30 + 50 = 80$

$80 < 100 \Rightarrow \text{True}$

$i \leftarrow 4$

if $80 + 40 \leq 100 \Rightarrow \text{No}$
goto else case.

$x[1] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$x[1] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$

$$x[i] = w - \text{weight} / w[i]$$

$$= \frac{100 - 80}{40} = \frac{20}{40} = \frac{1}{2}$$

$$\begin{aligned} \text{weight} &= \text{weight} + [x[i] * w[i]] \\ &= 80 + \frac{1}{2} * 40 \\ &= 100 \end{aligned}$$

$$x[i] \begin{bmatrix} 0 \\ 0 \\ 1 \\ \frac{1}{2} \\ 1 \end{bmatrix}$$

$$\underline{100 < 100 \Rightarrow \text{No}}$$

exit

Selected objects are 3rd, 5th and $\frac{1}{2}$ portion of 4th item.

Value / weight ratio method

Here object with high value / weight ratio is selected as best remaining object.

V	20	30	66	40	60
W	10	20	30	40	50
V/W	2	1.5	2.2	1	1.2

for $i \leftarrow 1$ to 5

$x[i] \leftarrow 0$

$x[i] \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

Weight $\leftarrow 0$

$0 < 100 \Rightarrow \text{True}$

$i \leftarrow 3$

$0 + 30 \leq 100 \Rightarrow \text{Yes}$

$x[3] \leftarrow 1$

Weight $\leftarrow 0 + 30 = 30$

$30 < 100 \Rightarrow \text{Yes}$

$i \leftarrow 1$

$30 + 10 \leq 100 \Rightarrow \text{Yes}$

$x[1] = 1$

$x[i] \begin{bmatrix} 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$x[i] \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$

$$\text{Weight} = 30 + 10 = 40$$

$$\underline{40 < 100} \Rightarrow \text{Yes}$$

$$i \leftarrow 2$$

$$\text{if } 40 + 20 \leq 100 \Rightarrow \text{Yes}$$

$$x[2] = 1$$

$$\text{Weight} = 40 + 20 = 60$$

$$\underline{60 < 100} \Rightarrow \text{Yes}$$

$$i \leftarrow 5$$

$$\text{if } 60 + 50 \leq 100 \Rightarrow \text{No}$$

go to else case

$$x[5] = \frac{100 - 60}{50} = \frac{4}{5}$$

$$\text{Weight} = 60 + \left[\frac{4}{5} * 50 \right]$$

$$= 60 + 40$$

$$= 100$$

$$\underline{100 < 100} \Rightarrow \text{No}$$

$$\text{exit } x[1] \begin{bmatrix} 1 \\ 1 \\ 1 \\ 0 \\ \frac{4}{5} \end{bmatrix}$$

Selected objects
all 1st, 2nd, 3rd &
 $\frac{4}{5}$ th portion of
5th object with
profit 164.

Q. Find the optimal solution to a knapsack

instance $n=8$, $m=110$,

$(p_1, p_2, \dots, p_8) = (11, 21, 31, 33, 43, 53, 55, 65)$

and

$(w_1, w_2, \dots, w_8) = (1, 11, 21, 33, 43, 53, 55, 65)$

Ans:

	1	2	3	4	5	6	7	8
P	11	21	31	33	43	53	55	65
W	1	11	21	33	43	53	55	65
P/W	11	1.9	1.47	1	1	1	1	1
Initial $x[i]$	0	0	0	0	0	0	0	0
$x[i]$	1	1	1	1	1	$\frac{1}{53}$		

$W_t \leftarrow 0$

* $0 < 110 \Rightarrow \text{True}$

$i \leftarrow 1$

$0 + 1 \leq 110 \Rightarrow \text{True}$

$x[1] \leftarrow 1$

$W_t \leftarrow 1$

* $1 < 110 \rightarrow \text{True}$

$$i \leftarrow 2$$

$$1 + 11 \leq 110 \rightarrow \text{True.}$$

$$a[2] \leftarrow 1$$

$$wt \leftarrow 1 + 11 \Rightarrow 12$$

$$* \quad 12 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 3$$

$$12 + 21 \leq 110 \rightarrow \text{True.}$$

$$a[3] \leftarrow 1$$

$$wt = 12 + 21 = 33$$

$$* \quad 33 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 4$$

$$33 + 33 \leq 110 \rightarrow \text{True.}$$

$$a[4] \leftarrow 1$$

$$wt \leftarrow 66.$$

$$* \quad 66 < 110 \rightarrow \text{True.}$$

$$i \leftarrow 5$$

$$66 + 43 \leq 110 \rightarrow \text{True.}$$

$$a[5] \leftarrow 1$$

$$wt \leftarrow 109.$$

$$* \quad 109 < 110 \rightarrow \text{True}$$

$$i \leftarrow 6$$

$$109 + 53 \leq 110 \rightarrow \text{False}$$

$$x[i] \leftarrow \frac{110 - 109}{53} = \frac{1}{53}$$

$$\text{hit} \leftarrow 109 + \frac{1}{53} * 53$$

$$\leftarrow 109 + 1 \Rightarrow 110$$

$$110 < 110 \rightarrow \text{False}$$

$\Rightarrow$ Exit.

$\Rightarrow$ Selected objects are 1st, 2nd, 3rd, 4th, 5th and $\frac{1}{53}$ rd fraction of 6th object.

Minimum Spanning Tree

A spanning tree T , for a graph $G: (V, E)$ is a subgraph $G' = (V', E')$ such that the following conditions hold.

1 $\Rightarrow$ The vertex set V' is same as that of graph G .

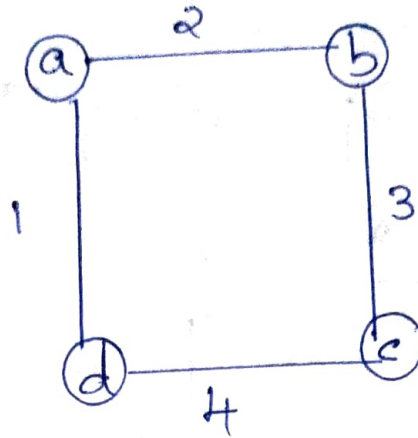
2 $\Rightarrow$ The edge set E' is a subset of E .

3 $\Rightarrow$ And there is no cycle.

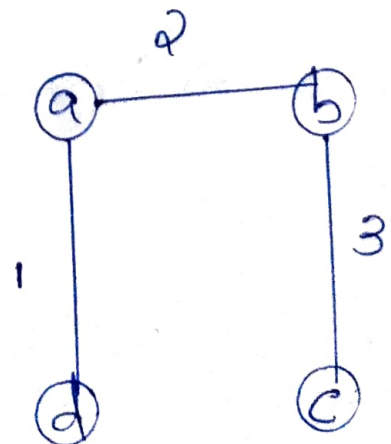
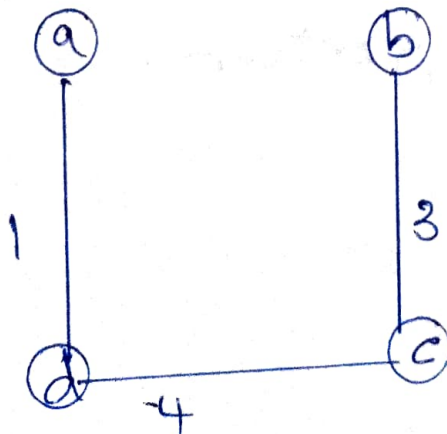
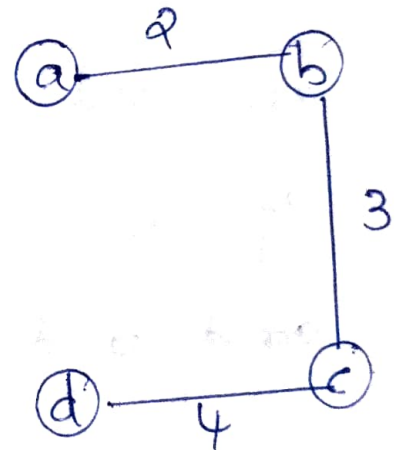
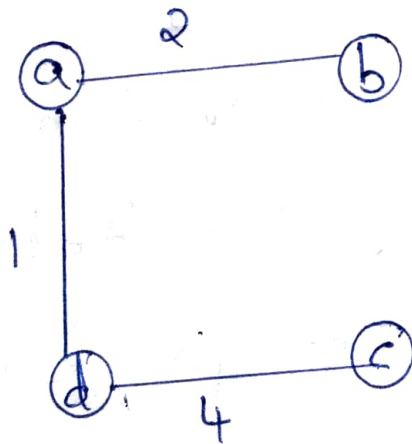
A minimum spanning tree [MST] for a weighted graph is a spanning tree with minimum weight.

For example :- Let $G = (V, E)$ is

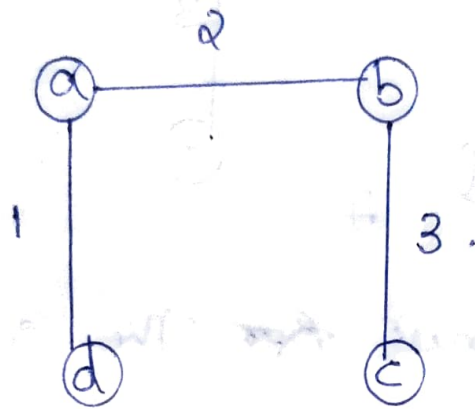
given



Spanning trees for the above graph
are shown below



In this, spanning tree with minimum weight is known as Minimum Spanning Tree.



Two different algorithms that are used to obtain minimum spanning tree of connected weighted and undirected graph are

⇒ Kruskal's algorithm, and

⇒ Prim's Algorithm.

Kruskal's Algorithm

This is one of the popular algorithms developed by Joseph Kruskal.

To create a minimum spanning tree using Kruskal's, we begin by choosing the edge with the minimum cost.

Algorithm works as follows:-

Suppose $G = (V, E)$ is a graph and T is a minimum spanning tree of graph G .

- 1: Initialize the spanning tree T to contain all the vertices in the graph G , but no edges.
- 2: choose the edge e with lowest weight from graph G .
- 3: check if both vertices from e are within the same set in the tree T .
⇒ If it is not present, add the edge e to the tree T and replace

the two sets that this edge connects

4: Delete the edge e from the graph G and repeat step 2 and 3 until there is no more edge to add or until the spanning tree T contains $(n-1)$ vertices

5: Exit.

Algorithm $\Rightarrow O(E \lg V)$

MST-Kruskals (G, w)

$A \leftarrow \emptyset$

for each vertex $v \in V(G)$

do makeset (v)

Sort the edges of E in non decreasing order by weight w .

for each edge $(u, v) \in E$, taken in non decreasing order by weight.

do if find-set $(u) \neq$ find-set (v)

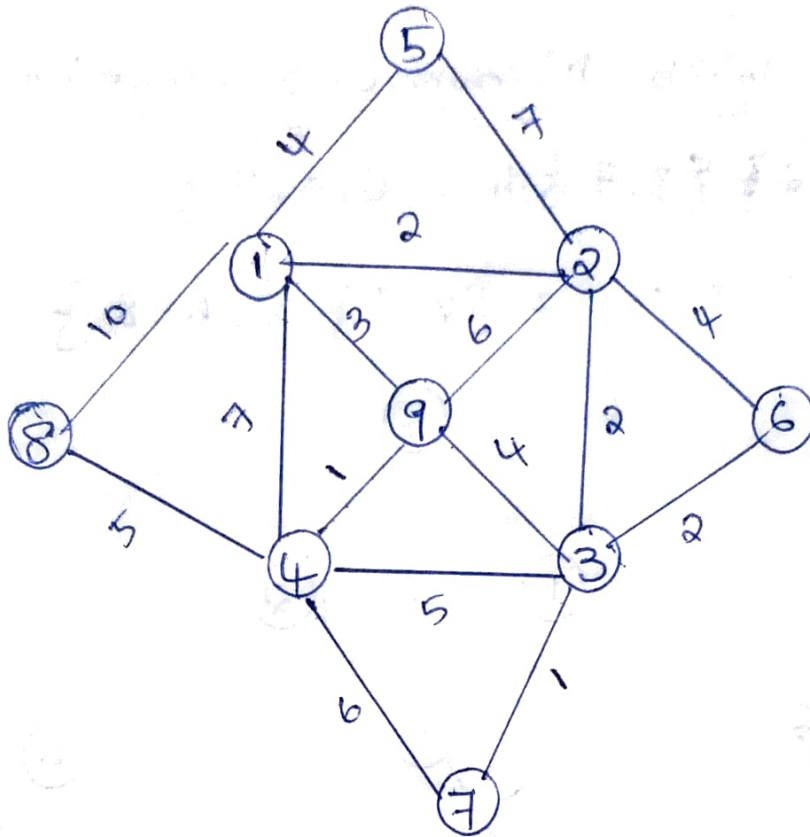
then $A \leftarrow A \cup \{(u, v)\}$

Union (u, v)

return A .

Example

Find the minimum spanning tree using Kruskal's algorithm for the graph given below



Solution

Add all vertices into the MST, π

~~Initialize the MST with all vertices~~

$\{\{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\} \{8\} \{9\}\}$

(5)

(1)

(2)

(8)

(9)

(6)

(3)

(4)

(7)

⇒ Edge with minimum weight is selected.

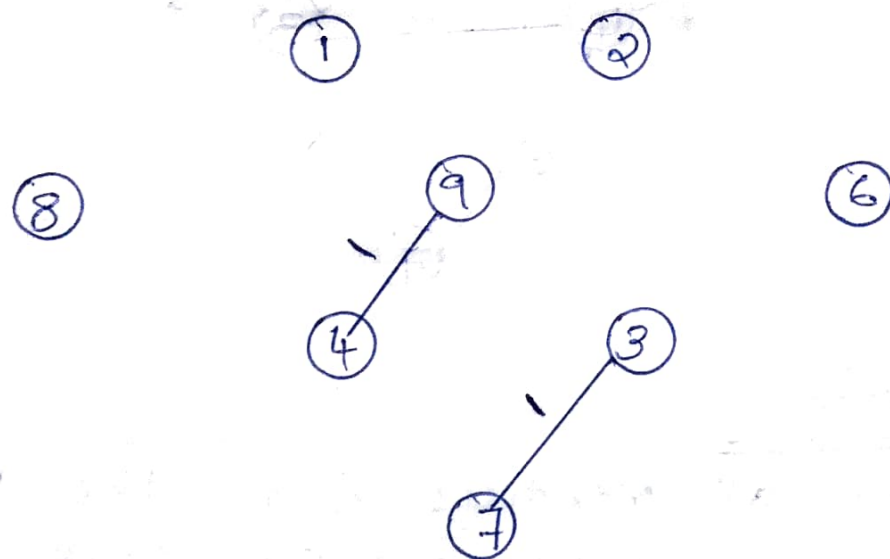
⇒ If it is not in same set, add it into same set.

Edge with Minimum Weight (7,3), (4,9)

$\{\{1\} \{2\} \{3,7\} 4,5,6,8,9\}$

$\{\{1\} \{2\} \{3,7\} \{4,9\} 5,6,8\}$

(5)



Minimum Weight edge (3,6), (3,2), (1,2)

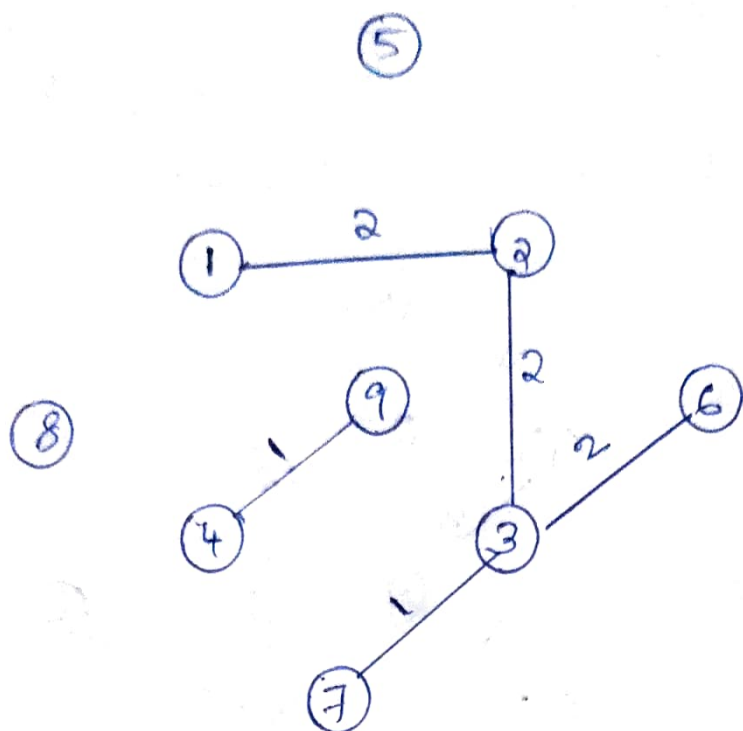
$\{\{3,7,6\} \{4,9\} 1,2,5,8\}$

⇒ (3,2)

$\{\{3,2,7,6\} \{4,9\} 1,5,8\}$

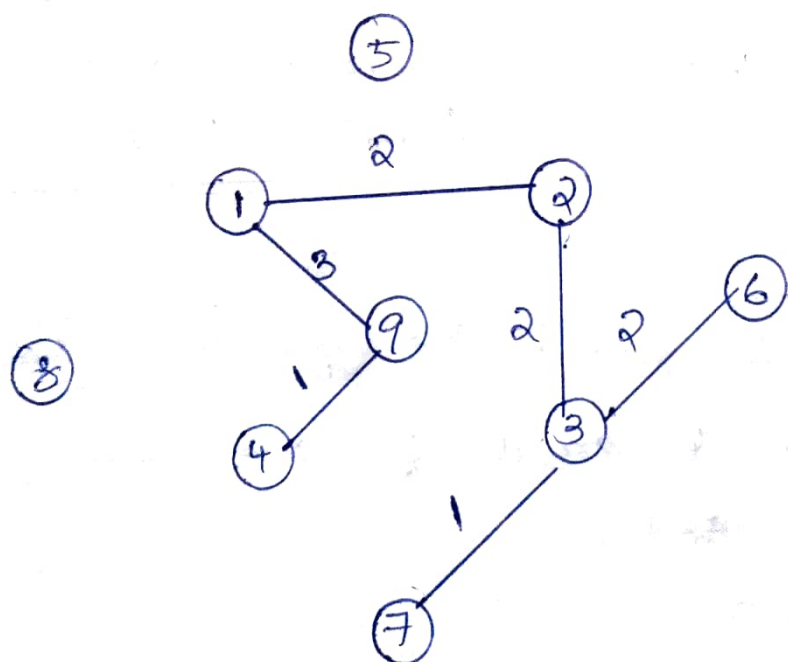
⇒ (1,2)

$\{\{1,2,3,6,7\} \{4,9\} 5,8\}$



Edge with minimum weight (1, 9)

$\{1, 2, 3, 6, 7, 4, 9\} \cup \{5, 8\}$



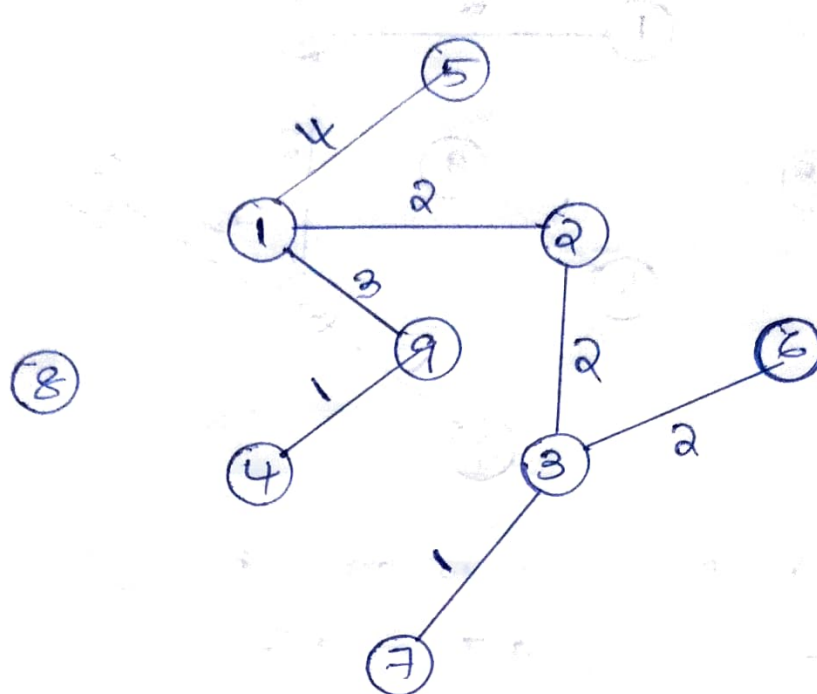
Edge with minimum weight (1, 5) (2, 6)

$\Rightarrow (1, 5)$

$\{1, 2, 3, 4, 5, 6, 7, 9\} \cup \{8\}$

$\Rightarrow (2, 6)$

$(2, 6)$ can't be joined because
2 and 6 are already in some set.



Edge with minimum weight $\Rightarrow$
 $(4, 3)$ $(4, 8)$

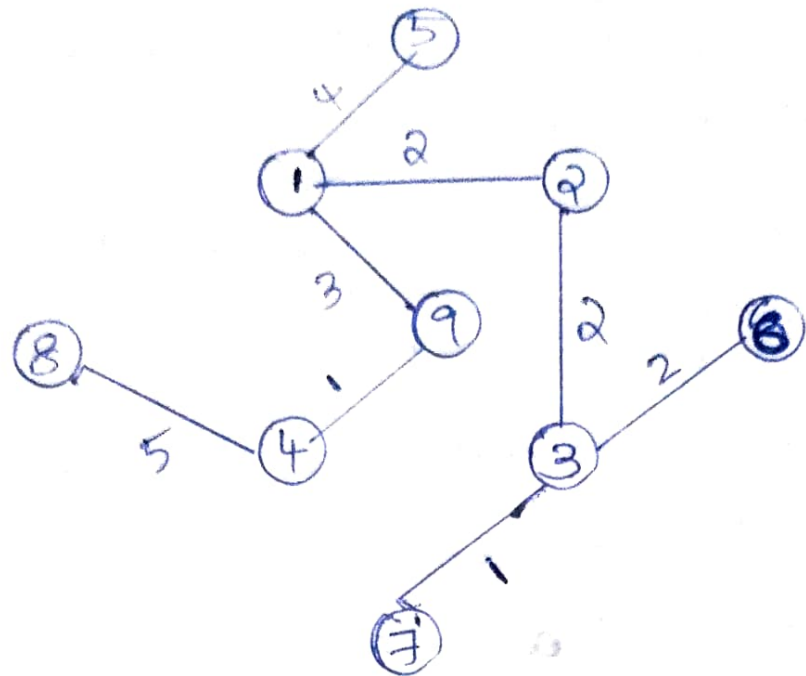
$\Rightarrow (4, 3)$

$(4, 3)$ can't be joined because
vertex 4 and 3 are already in
some set.

$\Rightarrow (4, 8)$

$\{1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Exit



This is the ~~minimum~~ spanning tree generated using Kruskal's algorithm.

Complexity of Kruskal's Algorithm.

⇒ To sort the edges in increasing order $\Rightarrow O(E \lg E)$

⇒ FindSet and Union operations take $O(E)$.

⇒ Along with the $|V|$ MakeSet operations these take total of $O(V + E)$ or $O(V)$ time.

⇒ Since $|E| \geq |V| - 1$ and $\alpha(|V|) = O(\log V) = O(\lg V)$

Running time is $O(E \lg V)$

PRIM'S ALGORITHM

This algorithm was discovered by Vojtech Jarník in 1936 and later rediscovered by Robert Prim.

Prim's algorithm constructs the minimum spanning tree edge by edge.

Prim's algorithm begins with a tree T , that contains a single vertex.

Prim's algorithm works as follows :-

1: Initially all nodes are made temporary

2: A node is arbitrarily selected as permanent.

3: Distance from permanent node to all other nodes are marked in the table.

4: A node with shorter distance is selected as permanent.

5: Repeat steps 3 and 4 until all nodes are made permanent.

Algorithm $\Rightarrow O(E + V \lg V)$

MST-prim's (G, w, r)

for each $u \in V(G)$

key $[u] \leftarrow \infty$

$\pi[u] \leftarrow \text{Nil}$

key $[r] \leftarrow 0$

$Q \leftarrow V[G]$

While $Q \neq \emptyset$

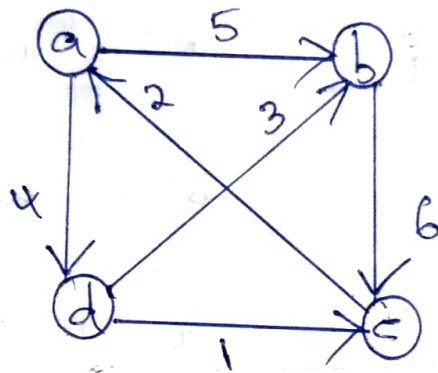
$u \leftarrow \text{Extract-Min}(Q)$

for each $v \in \text{Adj}[u]$

if $v \in Q$ and $w(u, v) < \text{key}[v]$
 then, $\pi[v] \leftarrow u$
 $\text{key}[v] \leftarrow w(u, v)$.

Example

Find minimum spanning tree T ,
 for the graph given using prims
 algorithm.



Solution

$\Rightarrow$ Initially all nodes are made
 temporary

Node	Status	Key	π
a	T	∞	Nil
b	T	∞	Nil
c	T	∞	Nil
d	T	∞	Nil

⇒ A node is arbitrarily selected as permanent

⇒ Then distance from permanent node to all other nodes are marked in the table.

Node	Status	key	π
a	P	0	0
b	T	5	a
c	T	∞	∞
d	T	4	a

⇒ A node with shorter distance is selected as permanent.

⇒ Repeat this until all nodes are permanent.

Node	Status	key	π
a	P	0	0
b	T	5	a
c	T	∞	∞
d	P	4	a

a	P	0	0
b	T	3	d
c	T	1	d
d	P	4	a

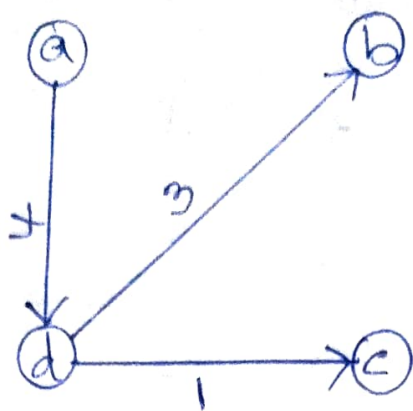
Make c Permanent

a	P	0	0
b	T	3	d
c	P	1	d
d	P	4	a

Make b permanent.

a	P	0	0
b	P	3	d
c	P	1	d
d	P	4	a

Construct Minimum spanning tree from this table.



⇒ In this example we didn't get the optimal solution.

⇒ Solution is near optimal.

This is because greedy algorithm may not give optimal solution always.

	Initial	a	d	c	b
a	∞	0	0	0	0
b	∞	5	3	3	3
c	∞	∞	1	1	1
d	∞	4	4	4	4

Using
Single
Table

Complexity of Prim's Algorithm.

⇒ steps 1-5 takes $O(V)$ time.

⇒ Therefore total time for all calls to Extract-min is $O(V \lg V)$.

⇒ steps 8-11 executes in $O(E)$ time.

⇒ step 11 altogether takes $O(\lg V)$

⇒ Therefore total time is

$$O(V \lg V + E \lg V) = O(E \lg V)$$

⇒ This can be improved by using Fibonacci heap into

$$\Rightarrow O(E + V \lg V).$$

Dijkstra's Algorithm.

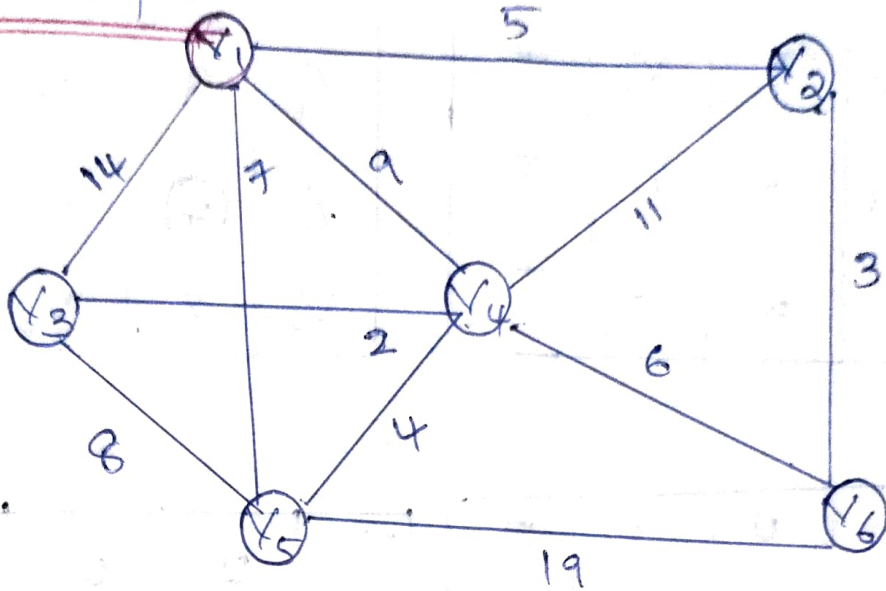
Dijkstra's algorithm solves the single source shortest path problem on a weighted, directed graph $G = (V, E)$ for the case in which all edge weights are non negative.

Procedure

- ① Initially make source node permanent and make it as the current working node. All other nodes are made to temporary.
- ② Examine all the temporary neighbours of the current working node and after checking the condition of minimum weight relabel the required node.
- ③ From all the temporary nodes find out the node which has minimum value of distance and make this node as our current working node.

- ④ Repeat steps ② and ③ until all the nodes are made permanent.
- ⑤ Stop.

Example



Find the shortest path from V_1 to V_6 .

①

Node	Distance	Predecessor	Status
V_1	0	0	P
V_2	∞	—	T
V_3	∞	—	T
V_4	∞	—	T
V_5	∞	—	T
V_6	∞	—	T

②

V_1	0	0	P
V_2	⑤	V_1	T
V_3	14	V_1	T
V_4	9	V_1	T
V_5	7	V_1	T
V_6	8	—	T

③

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	9	V_1	T
V_5	⑦	V_1	T
V_6	8	V_2	T

④

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	9	V_1	T
V_5	7	V_1	P
V_6	8	V_2	T

⑤

V_1	0	0	P
V_2	5	V_1	P
V_3	14	V_1	T
V_4	⑨	V_1	T
V_5	7	V_1	P
V_6	8	V_2	P

⑥

V_1	0	0	P
V_2	5	V_1	P
V_3	⑪	V_4	T
V_4	9	V_1	P
V_5	7	V_1	P
V_6	8	V_2	P

⑦

V_1	0	0	P
V_2	5	V_1	P
V_3	11	V_4	P
V_4	9	V_1	P
V_5	7	V_1	P
V_6	8	V_2	P

Shortest Paths

$$V_1 \longrightarrow V_2 \Rightarrow 5$$

$$V_1 \longrightarrow V_4 \longrightarrow V_3 \Rightarrow 11$$

$$V_1 \longrightarrow V_4 \Rightarrow 9$$

$$V_1 \longrightarrow V_5 \Rightarrow 7$$

$$V_1 \longrightarrow V_2 \longrightarrow V_6 \Rightarrow 8$$

Dijkstra's Algorithm

Dijkstra(G, w, v)

for each vertex $u \in V(G)$.

Key[u] $\leftarrow \infty$

$\pi[u] \leftarrow \text{Nil}$

Key[v] = 0

$Q \leftarrow V(G)$

while $Q \neq \emptyset$

$u \leftarrow \text{Extract-Min}(Q)$

remove u from Q .

for each $v \in \text{Adj}(u)$

alt = Key[u] + distance b/w
 (u, v)

if alt < Key[v]

~~alt~~ Key[v] = alt

$\pi[v] = u$.

Complexity Analysis of Dijkstra's Algorithm.

First Can

⇒ Dijkstra's algo works by performing 3 priority queue operations

- ① Insert
- ② Extract-Min, and
- ③ Decrease Key.

⇒ Algorithm calls both Insert and Extract-Min once per vertex, and Decrease key at most $|E|$ times overall.

⇒ Each Insert and Decrease key takes $O(1)$ time, and each Extract-min operation takes $O(V)$ time.

Total time ⇒ $O(V^2 + E) = O(V^2)$

Second case

If the graph is sufficiently sparse we can improve the algorithm by implementing min-priority queue with binary min heap.

⇒ Each Extract-min operation takes $O(\lg V)$ time.

⇒ Time to build binary min heap is $O(V)$.

⇒ Each decrease key operation takes $O(\lg V)$ time and there are at most $|E|$ such operations.

⇒ Therefore the total running time is $O((V+E)\lg V)$

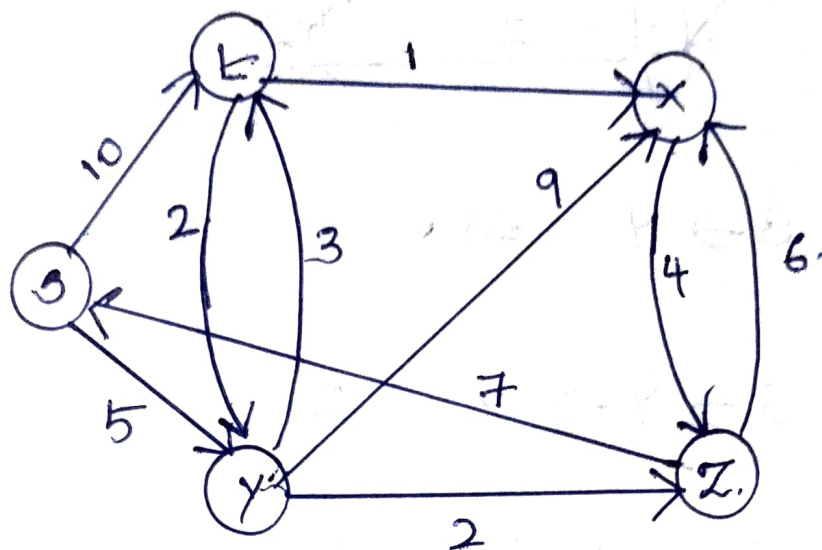
⇒ $O(E \lg V)$ if all vertices

are reachable from the source.

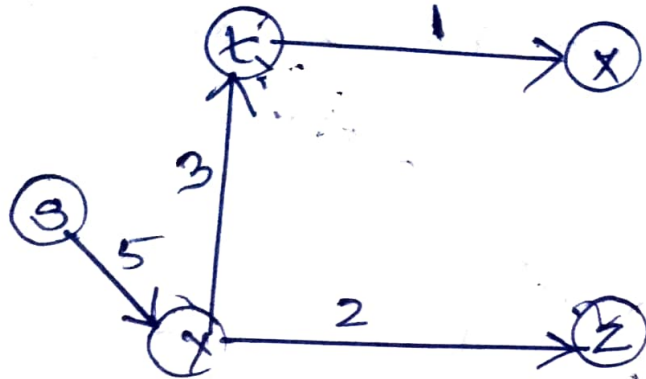
Third case

⇒ We can achieve a running time of $O(V \lg V + E)$ by implementing min-priority queue with fibonacci heap.

Q. Find out the shortest path to the graph shown below using Dijkstra's Algorithm.



	S	y	z	t	x
S	0	0	0	0	0
t	10	8	8	8	8
x	-	14	13	9	9
y*	5	5	5	5	5
z*	-	7	7	7	7



$$s \rightarrow y \Rightarrow 5$$

$$s \rightarrow y \rightarrow z \Rightarrow 7$$

$$s \rightarrow y \rightarrow t \Rightarrow 8$$

$$s \rightarrow y \rightarrow t \rightarrow x \Rightarrow 9$$